

Spring Boot Monolithique vs Microservices Spring Boot

TaskManager Microservices

Présentation

Dans cette vidéo, nous allons découvrir les différences entre une application Spring Boot classique (monolithique) et une architecture Microservices Spring Boot.

Pour illustrer ces concepts, nous utiliserons le projet :

TaskManager Microservices

Objectifs :

- Comprendre l'architecture monolithique
- Comprendre les principes des microservices
- Découvrir les composants Spring Cloud
- Analyser une architecture microservices réelle
- Identifier les avantages et les inconvénients de chaque approche

1. Qu'est-ce qu'une application Spring Boot classique ?

Une application Spring Boot classique est généralement développée sous la forme d'un monolithe.

Toutes les fonctionnalités sont regroupées dans une seule application :

- Gestion des utilisateurs
- Gestion des tâches
- Authentification
- Notifications
- Accès aux données

Architecture simplifiée :

TaskManager

- Controllers
- Services
- Repositories
- Base de données unique

Toutes les fonctionnalités sont déployées ensemble.

Avantages du monolithe

- Développement rapide
- Architecture simple
- Débogage facile
- Déploiement unique
- Faible complexité opérationnelle

Inconvénients du monolithe

- Difficulté de maintenance lorsque l'application grandit
- Déploiement complet à chaque modification
- Scalabilité limitée
- Couplage fort entre les composants
- Risque accru d'impact lors des évolutions

2. Pourquoi passer aux microservices ?

Lorsqu'une application devient importante, il devient difficile de gérer l'ensemble dans un seul projet. L'objectif des microservices est de découper l'application en plusieurs services indépendants.

Chaque service :

- Possède sa responsabilité propre

- Peut être développé indépendamment
- Peut être déployé indépendamment
- Peut être mis à l'échelle indépendamment

3. Présentation du projet TaskManager Microservices

Le projet TaskManager Microservices met en œuvre une architecture moderne basée sur Spring Boot et Spring Cloud.

Architecture générale :

Client



API Gateway



Services métiers

- User Service
- Task Service
- Auth Service



Infrastructure

- Eureka Discovery Server
- Config Server
- Bases de données

4. Composants principaux de l'architecture

API Gateway

La Gateway constitue le point d'entrée unique du système.

Rôles :

- Routage des requêtes
- Sécurisation
- Filtrage
- Centralisation des accès

Avantages :

- Réduction de la complexité côté client
- Contrôle centralisé des flux

Eureka Discovery Server

Dans une architecture microservices, les services doivent se découvrir automatiquement.

Eureka permet :

- L'enregistrement des services
- La découverte dynamique des services
- La tolérance aux changements d'adresses

Chaque service s'enregistre automatiquement auprès d'Eureka.

Spring boot

Spring Cloud complète Spring Boot en apportant toutes les fonctionnalités nécessaires à une architecture microservices.

Dans notre projet TaskManager, il permet la découverte des services avec Eureka, la centralisation des accès grâce à l'API Gateway et la gestion centralisée de la configuration avec Config Server. Il simplifie ainsi le développement, le déploiement et la maintenance d'applications distribuées modernes.

User Service

Responsabilités :

- Gestion des utilisateurs
- Création des comptes
- Consultation des informations utilisateur

Le service est autonome et indépendant.

Task Service

Responsabilités :

- Création des tâches
- Mise à jour des tâches
- Suppression des tâches
- Consultation des tâches

Ce service se concentre uniquement sur la gestion métier des tâches.

Authentication Service

Responsabilités :

- Authentification
- Gestion des tokens
- Sécurisation des accès

Séparation claire de la logique de sécurité.

Config Server

Le Config Server centralise la configuration de tous les services.

Avantages :

- Gestion centralisée
- Réduction des duplications
- Modification simplifiée des paramètres

5. Communication entre les microservices

Les microservices doivent communiquer entre eux.

Plusieurs solutions sont possibles :

- RestTemplate
- WebClient
- OpenFeign

Objectifs :

- Échange de données
- Collaboration entre services
- Conservation de l'autonomie de chaque service

6. Comparaison Monolithe vs Microservices

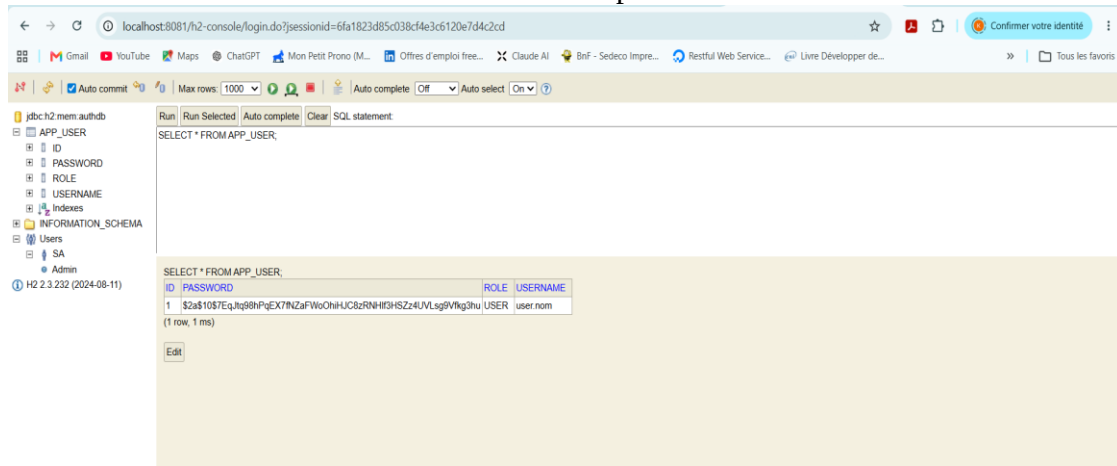
Critère	Monolithe	Microservices
Développement initial	Plus rapide	Plus complexe
Déploiement	Unique	Multiple
Scalabilité	Limitée	Très élevée
Maintenance	Difficile à grande échelle	Plus simple
Résilience	Faible	Meilleure
Complexité technique	Faible	Élevée

//

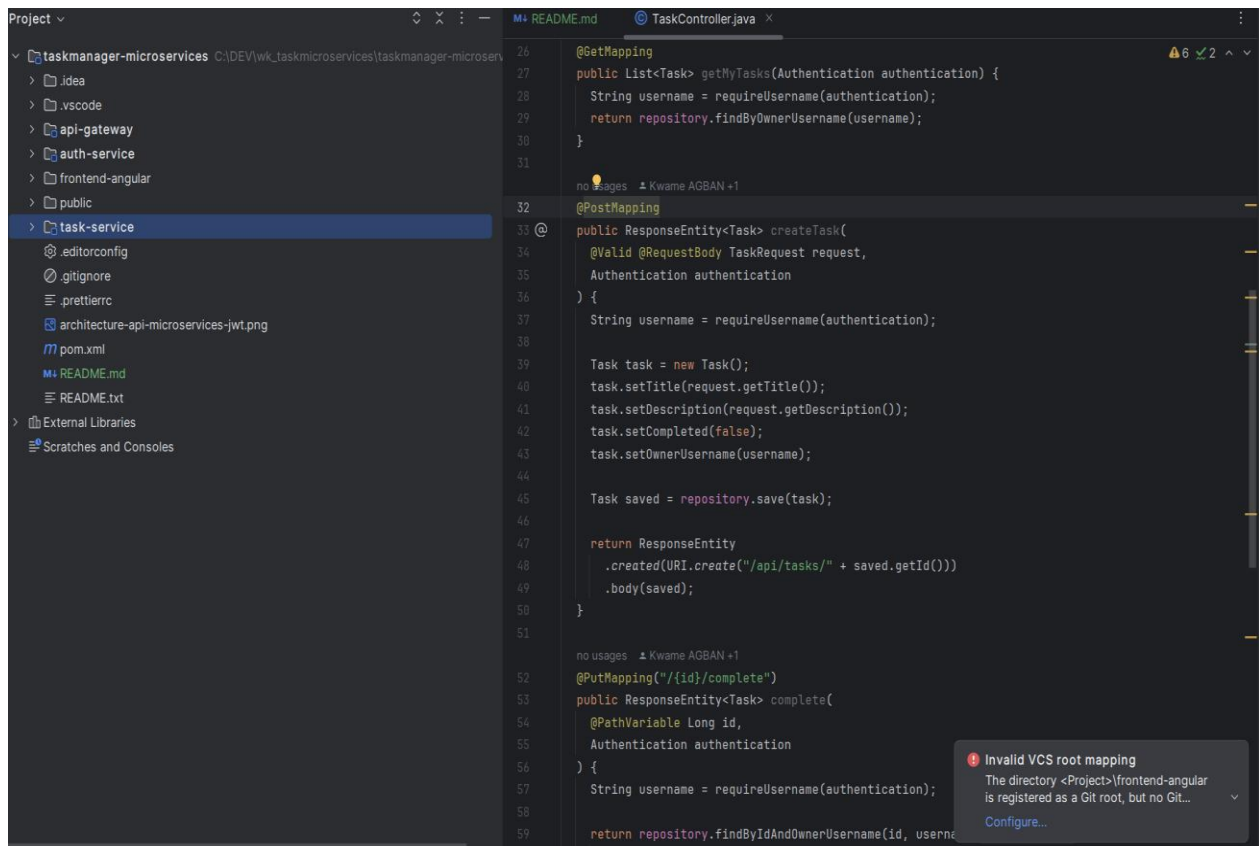
Présentation de l'application TaskManager

Application développée en Spring boot, spring cloud, Angular, typeScript et base H2

Base de données utilisée : H2 dont l'url est : http://localhost:8081/h2-console



Code Java sous IntelliJ



Ecrans de Gestion des tâches

- Connexion utilisateur

Connexion

Nom d'utilisateur

user.nom

Mot de passe

.....

Se connecter

- Gestion des tâches

Mes tâches

Déconnexion

Titre de la tâche

Description

Ajouter

task1

Description 1

Statut : En cours

Terminer

Supprimer

task 2

Description 2

Statut : En cours

Terminer

Supprimer

Conclusion

Spring Boot reste une excellente solution pour démarrer rapidement un projet.

Cependant, lorsque l'application grandit et nécessite davantage de scalabilité et d'autonomie, l'architecture microservices devient une solution pertinente.

Le projet TaskManager Microservices illustre concrètement comment construire une architecture moderne basée sur :

- Spring Boot
- Spring Cloud
- Eureka
- API Gateway
- Config Server

Merci d'avoir suivi cette présentation.

N'hésitez pas à consulter le dépôt GitHub du projet et à partager vos retours en commentaire.